LAB ASSIGNMENT-8.1

Humera Mahaveen
2303A53030
Batch-46

Task 1: Password Strength Validator

Prompt

Generate assert test cases for a Python function is_strong_password(password) that validates password strength. Conditions: minimum 8 characters, must include uppercase, lowercase, digit, special character, and no spaces.

Code

```
import re

def is_strong_password(password):
    if len(password) < 8 or " " in password:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"[0-9]", password):
        return False
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return False
    return True

# Test Cases
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == False
```

Task 2: Number Classification

Prompt

Generate assert test cases for a function classify_number(n) that classifies numbers as Positive, Negative, or Zero and handles invalid inputs like strings or None.

Code

```
def classify_number(n):
    if not isinstance(n, (int, float)):
        return "Invalid Input"
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    else:
        return "Zero"

# Test Cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number("abc") == "Invalid Input"
```

Task 3: Anagram Checker

Prompt

Generate assert test cases for a function is_anagram(str1, str2) that checks if two strings are anagrams, ignoring case, spaces, and punctuation.

Code

```
def is_anagram(str1, str2):
    def clean(s):
        return ''.join(c.lower() for c in s if c.isalnum())
    return sorted(clean(str1)) == sorted(clean(str2))

# Test Cases
assert is_anagram("listen", "silent") == True
```

```
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

Task 4: Inventory Class

Prompt

Generate assert-based test cases for a Python class Inventory with methods: add_item(name, quantity), remove_item(name, quantity), and get_stock(name).

Code

```
class Inventory:
    def __init__(self):
        self.stock = {}

    def add_item(self, name, quantity):
        self.stock[name] = self.stock.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        if name in self.stock and self.stock[name] >= quantity:
            self.stock[name] -= quantity

    def get_stock(self, name):
        return self.stock.get(name, 0)

# Test Cases
inv = Inventory()
inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
```

Task 5: Date Validation & Formatting

Prompt

Generate assert test cases for a function validate_and_format_date(date_str) that validates "MM/DD/YYYY" format, handles invalid dates, and converts valid dates to "YYYY-MM-DD".

Code

```
from datetime import datetime

def validate_and_format_date(date_str):
    try:
        date = datetime.strptime(date_str, "%m/%d/%Y")
        return date.strftime("%Y-%m-%d")
    except:
        return "Invalid Date"

# Test Cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```